MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Bulimar Rodion FAF-242

Report

Laboratory Work 4
Block Ciphers. The DES Algorithm

Checked by:
Zaica Maia, *university assistant*
FCIM, UTM

Chișinău – 2026

1 Purpose

The purpose of this laboratory work is to study the Data Encryption Standard (DES) block cipher by implementing, step by step, all of its building blocks: the key schedule, the initial and final permutations, the expansion function E, the eight S-boxes and the permutation P. The program (`lab4.py`) implements all eleven tasks 2.1–2.11 of the assignment, so that any task selected by the formula nr_sarcina $= n \bmod 11$ can be demonstrated. For every task it prints the tables used and all intermediate values, and the input can be typed by the user or generated randomly. Additionally, a complete DES encryption and decryption of a block and of a text (with padding) is provided.

2 Short Theory

DES is a symmetric Feistel block cipher. It processes a 64-bit block with a 64-bit key, of which only 56 bits are used (every eighth bit is a parity bit and is discarded by the first key permutation). The data pass through an initial permutation IP, 16 identical Feistel rounds, a swap of the halves and the final permutation IP^{-1}.

Key schedule. The key K is reduced to 56 bits by $K^+ = PC\text{-}1(K)$ and split into two 28-bit halves $C_0D_0 = K^+$. For $i = 1, \ldots, 16$:

$$C_i = LS_i(C_{i-1}), \qquad D_i = LS_i(D_{i-1}), \qquad K_i = PC\text{-}2(C_iD_i),$$

where LS_i is a cyclic left shift by 1 position for rounds $i \in \{1, 2, 9, 16\}$ and by 2 positions for all the other rounds, and K_i has 48 bits.

Encryption. After $IP(M) = L_0R_0$ (32 bits each), each round computes:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

$$f(R, K) = P\big(S_1(B_1)S_2(B_2)\ldots S_8(B_8)\big), \qquad B_1B_2\ldots B_8 = K \oplus E(R).$$

Each B_j has 6 bits. For $B_j = b_1b_2b_3b_4b_5b_6$ the row of S_j is given by b_1b_6, the column by $b_2b_3b_4b_5$, and the output is a 4-bit number. The ciphertext block is

$$C = IP^{-1}(R_{16}L_{16}),$$

i.e. the halves are swapped before the final permutation. Decryption uses the same algorithm with the round keys applied in the reverse order $K_{16}, \ldots, K_1$.

DES with a 56-bit key is considered insecure today (it can be exhaustively searched), and electronic codebook (ECB) mode, which encrypts every block independently, leaks repeated plaintext blocks. The implementation is therefore intended for educational purposes only.

3 Program Implementation

The program is written in Python and is organised in layers: constant tables, bit-string helpers, the key schedule, the round function, one function per assignment task, full-block and full-text DES, and a menu. Bits are stored as strings of `'0'` and `'1'`, which makes every permutation a simple lookup and makes all intermediate values easy to print. Table 1 maps each task of the assignment to the function that solves it.

Task	Function	Computes
2.1	`task_2_1`	K^+ from an 8-character key
2.2	`task_2_2`	C_i, D_i for a given i
2.3	`task_2_3`	round key K_i for a given i
2.4	`task_2_4`	L_1 (equal to R_0) from an 8-character message
2.5	`task_2_5`	all round keys $K_1, \ldots, K_{16}$
2.6	`task_2_6`	$B_1B_2 \ldots B_8 = K_i \oplus E(R_{i-1})$
2.7	`task_2_7`	$S_1(B_1) \ldots S_8(B_8)$
2.8	`task_2_8`	R_k from L_{k-1} and the S-box output
2.9	`task_2_9`	$S_j(B_j)$ for a given j
2.10	`task_2_10`	R_i from the S-box output and L_{i-1}
2.11	`task_2_11`	encrypted block from L_{16}, R_{16}
bonus	`task_full`, `task_full_text`, `task_full_decrypt`	full DES of a block, of a text, and decryption

Table 1: Mapping between the tasks and the functions of `lab4.py`

3.1 Constant tables

```
IP = [58,50,42,34,26,18,10,2, 60,52,44,36,28,20,12,4,
      62,54,46,38,30,22,14,6, 64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,  59,51,43,35,27,19,11,3,
      61,53,45,37,29,21,13,5, 63,55,47,39,31,23,15,7]
FP = [40,8,48,16,56,24,64,32, 39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30, 37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28, 35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26, 33,1,41,9,49,17,57,25]
E = [32,1,2,3,4,5, 4,5,6,7,8,9, 8,9,10,11,12,13, 12,13,14,15,16,17,
     16,17,18,19,20,21, 20,21,22,23,24,25, 24,25,26,27,28,29, 28,29,30,31,32,1]
P = [16,7,20,21,29,12,28,17, 1,15,23,26,5,18,31,10,
     2,8,24,14,32,27,3,9, 19,13,30,6,22,11,4,25]
PC1 = [57,49,41,33,25,17,9, 1,58,50,42,34,26,18, 10,2,59,51,43,35,27,
       19,11,3,60,52,44,36, 63,55,47,39,31,23,15, 7,62,54,46,38,30,22,
       14,6,61,53,45,37,29, 21,13,5,28,20,12,4]
PC2 = [14,17,11,24,1,5, 3,28,15,6,21,10, 23,19,12,4,26,8, 16,7,27,20,13,2,
       41,52,31,37,47,55, 30,40,51,45,33,48, 44,49,39,56,34,53, 46,42,50,36,29,32]
SHIFTS = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
```

Comment: The tables IP, IP^{-1} (`FP`), E, P, PC-1, PC-2 and the list of shifts `SHIFTS` are stored exactly as in FIPS 46-3. The eight S-boxes are stored in the list `S` as $8 \times 4 \times 16$ integers (lines 22–39 of the source). Each task prints the tables it uses by calling `show_table` or `show_sboxes`.

3.2 Bit helpers

```
def group(bits, n):
    return " ".join(bits[i:i + n] for i in range(0, len(bits), n))

def permute(bits, table):
    return "".join(bits[i - 1] for i in table)

def xor(a, b):
    return "".join("1" if x != y else "0" for x, y in zip(a, b))

def lshift(bits, n):
    return bits[n:] + bits[:n]

def text_to_bits(text):
    return "".join(format(ord(c), "08b") for c in text)

def bits_to_hex(bits):
    return format(int(bits, 2), "0%dX" % (len(bits) // 4))
```

Comment: `permute` applies any permutation table (indices are 1-based, as in the standard), `xor` adds two bit strings modulo 2, `lshift` performs the cyclic left shift used in the key schedule,

`text_to_bits` converts each ASCII character to 8 bits, and `bits_to_hex` prints a bit string in hexadecimal. `group` only inserts spaces for readable output.

3.3 User input and random generation

The functions `ask_bits`, `ask_text8`, `ask_int` and `ask_ciphertext_bits` read and validate the data. A value can be typed in binary or in hexadecimal (prefix `0x`); entering `r` generates a random value of the required length. Lengths are checked (for example, exactly 56 bits for K^+ or exactly 8 ASCII characters for a key), and the user is asked again on an invalid value.

3.4 Key schedule (tasks 2.1, 2.2, 2.3, 2.5)

```
def compute_kplus(key_bits, verbose=True):
    kp = permute(key_bits, PC1)
    if verbose:
        show_table("PC-1", PC1, 7)
        print(f"\nCheia K (64 biti)  : {group(key_bits, 8)}")
        print(f"K+ = PC-1(K) (56 b): {group(kp, 7)}")
        print(f"K+ (hex)           : {bits_to_hex(kp)}")
    return kp

def compute_CD(kp, upto, verbose=True):
    C, D = kp[:28], kp[28:]
    if verbose:
        print(f"\nC0 = {group(C, 4)}\nD0 = {group(D, 4)}")
    CD = [(C, D)]
    for i in range(1, upto + 1):
        C, D = lshift(C, SHIFTS[i - 1]), lshift(D, SHIFTS[i - 1])
        CD.append((C, D))
        if verbose:
            print(f"C{i:<2} = {group(C, 4)}   (shift stanga cu {SHIFTS[i - 1]})")
            print(f"D{i:<2} = {group(D, 4)}")
    return CD

def round_key(C, D):
    return permute(C + D, PC2)

def all_round_keys(kp, verbose=False):
    CD = compute_CD(kp, 16, verbose)
    return [round_key(*CD[i]) for i in range(1, 17)]
```

Comment: `compute_kplus` applies PC-1 to the 64-bit key. `compute_CD` splits K^+ into C_0 and D_0 and applies the cyclic shifts from `SHIFTS` up to the requested round, printing every C_i and D_i. `round_key` applies PC-2 to the concatenation C_iD_i, and `all_round_keys` returns $K_1, \ldots, K_{16}$. Tasks 2.1, 2.2, 2.3 and 2.5 are thin wrappers that read the input, call these functions and print the tables and results.

3.5 S-box lookup

```
def sbox(j, six):
    row = int(six[0] + six[5], 2)
    col = int(six[1:5], 2)
    val = S[j][row][col]
    return row, col, val, format(val, "04b")
```

Comment: The first and last bits of the 6-bit block select the row, the four middle bits select the column. The function returns the row, the column, the table value and its 4-bit representation, so that tasks 2.7 and 2.9 can show the full lookup.

3.6 Message, round and S-box tasks (tasks 2.4, 2.6–2.10)

```
def task_2_4():
    m = ask_text8("Mesajul")
```

```
    mb = text_to_bits(m)
    print("\nMesaj in binar:")
    for c in m:
        print(f"  '{c}' = {format(ord(c), '08b')}")
    show_table("IP", IP, 8)
    ip = permute(mb, IP)
    L0, R0 = ip[:32], ip[32:]
    print(f"\nM        = {group(mb, 8)}")
    print(f"IP(M)    = {group(ip, 8)}")
    print(f"L0 = {group(L0, 8)}\nR0 = {group(R0, 8)}")
    print(f"\nL1 = R0 = {group(R0, 8)}   (hex: {bits_to_hex(R0)})")
```

```
def task_2_6():
    i = ask_int("Runda i", 1, 16)
    ki = ask_bits(f"K{i}", 48)
    r = ask_bits(f"R{i - 1}", 32)
    show_table("E (expansiune 32 -> 48)", E, 6)
    er = permute(r, E)
    x = xor(ki, er)
    print(f"\nR{i - 1}          = {group(r, 8)}")
    print(f"E(R{i - 1})       = {group(er, 6)}")
    print(f"K{i}             = {group(ki, 6)}")
    print(f"K{i} + E(R{i - 1}) = {group(x, 6)}")
    print("\nB1B2...B8:")
    for j in range(8):
        print(f"  B{j + 1} = {x[6 * j:6 * j + 6]}")
```

Comment: Task 2.4 converts the message to bits, applies IP, splits the result into L_0 and R_0 and shows that $L_1 = R_0$. Task 2.6 expands R_{i-1} with E, adds K_i modulo 2 and splits the 48-bit result into $B_1, \ldots, B_8$. Tasks 2.7 and 2.9 perform the S-box lookups for all eight blocks or for one given block B_j, respectively.

```
def task_2_8():
    k = ask_int("Runda k", 1, 16)
    l = ask_bits(f"L{k - 1}", 32)
    s = ask_bits("Rezultatul cutiilor S (32 biti)", 32)
    show_table("P", P, 8)
    ps = permute(s, P)
    r = xor(l, ps)
    print(f"\nS-out        = {group(s, 8)}")
    print(f"P(S-out)     = {group(ps, 8)}")
    print(f"L{k - 1}         = {group(l, 8)}")
    print(f"R{k} = L{k - 1} + P(S-out) = {group(r, 8)}   (hex: {bits_to_hex(r)})")
```

Comment: Tasks 2.8 and 2.10 apply the permutation P to the concatenated S-box outputs and add the result to L_{i-1} modulo 2, obtaining R_i. They differ only in the way in which the data are presented (the S-box output is printed as 32 bits in task 2.8 and as eight 4-bit groups in task 2.10).

3.7 Final permutation (task 2.11)

```
def task_2_11():
    l16 = ask_bits("L16", 32)
    r16 = ask_bits("R16", 32)
    show_table("IP^-1 (permutarea finala)", FP, 8)
    pre = r16 + l16
    c = permute(pre, FP)
    print(f"\nL16 = {group(l16, 8)}\nR16 = {group(r16, 8)}")
    print(f"R16L16 (se inverseaza!) = {group(pre, 8)}")
    print(f"C = IP^-1(R16L16)       = {group(c, 8)}")
    print(f"\nBloc criptat (hex): {bits_to_hex(c)}")
```

Comment: The halves are swapped to $R_{16}L_{16}$ and IP^{-1} is applied. The program explicitly prints this swap, because forgetting it is the most common error in this step.

3.8 Complete DES for a block and for a text

```
def feistel(r, ki, verbose=False):
    er = permute(r, E)
    x = xor(ki, er)
    out = ""
    for j in range(8):
        out += sbox(j, x[6 * j:6 * j + 6])[3]
    return permute(out, P)

def des_encrypt_block(block_bits, key_bits, rounds=16, verbose=False):
    keys = all_round_keys(compute_kplus(key_bits, False))
    ip = permute(block_bits, IP)
    L, R = ip[:32], ip[32:]
    if verbose:
        print(f"IP = {group(ip, 8)}\nL0 = {group(L, 8)}\nR0 = {group(R, 8)}")
    for i in range(1, rounds + 1):
        L, R = R, xor(L, feistel(R, keys[i - 1]))
        if verbose:
            print(f"Runda {i:2d}: K={bits_to_hex(keys[i - 1])} L={bits_to_hex(L)} R={bits_to_hex(R)}")
    return permute(R + L, FP)

def des_decrypt_block(block_bits, key_bits, verbose=False):
    keys = list(reversed(all_round_keys(compute_kplus(key_bits, False))))
    ip = permute(block_bits, IP)
    L, R = ip[:32], ip[32:]
    if verbose:
        print(f"IP = {group(ip, 8)}\nL16 = {group(L, 8)}\nR16 = {group(R, 8)}")
    for round_index, key in enumerate(keys, 1):
        L, R = R, xor(L, feistel(R, key))
        if verbose:
            print(f"Runda inversa {round_index:2d}: K={bits_to_hex(key)} "
                  f"L={bits_to_hex(L)} R={bits_to_hex(R)}")
    return permute(R + L, FP)
```

Comment: `feistel` implements $f(R, K) = P(S(K \oplus E(R)))$. `des_encrypt_block` runs IP, the rounds and IP^{-1}; its parameter `rounds` allows a truncated version of the algorithm. Decryption is the same Feistel network with the reversed list of round keys.

```
def pkcs7_pad(text_bits):
    remainder = len(text_bits) % 64
    padding_bits = 64 - remainder if remainder else 64
    padding_byte = padding_bits // 8
    return text_bits + format(padding_byte, "08b") * padding_byte

def pkcs7_unpad(text_bits):
    if not text_bits or len(text_bits) % 64 != 0 or len(text_bits) == 0:
        raise ValueError("lungimea textului nu este multiplu de bloc")
    padding_byte = int(text_bits[-8:], 2)
    if padding_byte < 1 or padding_byte > 8:
        raise ValueError("padding-ul DES nu este valid")
    padding = format(padding_byte, "08b") * padding_byte
    if text_bits[-padding_byte * 8:] != padding:
        raise ValueError("padding-ul DES nu este valid")
    return text_bits[:-padding_byte * 8]
```

Comment: For texts longer than one block, `des_encrypt_text` splits the message into 64-bit blocks and encrypts them independently (ECB). The last block is completed with PKCS#7 padding: p bytes of value p are appended, and a full block of padding is added if the length is already a multiple of 8 bytes. `pkcs7_unpad` checks that the padding is valid before removing it.

3.9 Self-test and menu

```
def self_test():
    k = format(0x133457799BBCDFF1, "064b")
    m = format(0x0123456789ABCDEF, "064b")
    got = bits_to_hex(des_encrypt_block(m, k))
    decrypted = bits_to_hex(des_decrypt_block(format(int(got, 16), "064b"), k))
    print("Self-test criptare:",
          "OK" if got == "85E813540F0AB405" else f"EROARE ({got})")
    print("Self-test decriptare:",
```

```
9           "OK" if decrypted == "0123456789ABCDEF" else f"EROARE ({decrypted})")
```

Comment: At every start the program encrypts the well-known test vector (key `133457799BBCDFF1`, plaintext `0123456789ABCDEF`), compares the result with the expected ciphertext `85E813540F0AB405` and then decrypts it back. The menu in `main` lists tasks 2.1–2.11 and the three additional options, and repeats until the user chooses 0.

4 Results

4.1 Validation of the implementation

The program's self-test reports success for both directions:

```
Self-test criptare: OK
Self-test decriptare: OK
```

In addition, the functions `des_encrypt_text` and `des_decrypt_text` were compared with the DES implementation of the `pycryptodome` library (ECB mode with PKCS#7 padding) on 200 random keys and texts of 0–40 characters. The ciphertexts were identical in all 200 cases, and decryption recovered the original text each time.

All task examples below use the classical worked example of DES (key `133457799BBCDFF1`, message `0123456789ABCDEF`), which makes each intermediate value easy to check against the literature. Inputs were given in hexadecimal.

4.2 Task 2.1: K^+ from an 8-character key

The key `Cripto26` was entered. The program shows the ASCII codes, the table PC-1 and K^+.

```
Cheia DES [8 caractere ASCII; 'r' = aleator]: Cripto26

Caractere -> cod ASCII pe 8 biti:
  'C' =  67 = 01000011
  'r' = 114 = 01110010
  'i' = 105 = 01101001
  'p' = 112 = 01110000
  't' = 116 = 01110100
  'o' = 111 = 01101111
  '2' =  50 = 00110010
  '6' =  54 = 00110110

Tabelul PC-1:
 57 49 41 33 25 17  9
  1 58 50 42 34 26 18
 10  2 59 51 43 35 27
 19 11  3 60 52 44 36
 63 55 47 39 31 23 15
  7 62 54 46 38 30 22
 14  6 61 53 45 37 29
 21 13  5 28 20 12  4

Cheia K (64 biti)  : 01000011 01110010 01101001 01110000 01110100 01101111 00110010 00110110
K+ = PC-1(K) (56 b): 0000000 0001111 1111111 1101101 1110001 1101100 0000100 1001010
K+ (hex)           : 003FFEDE3B024A
```

4.3 Task 2.2: C_i and D_i for a given i

For $K^+ =$ `F0CCAAF556678F` and $i = 3$:

```
K+ [56 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0xF0CCAAF556678F
i [0..16]: 3

K+ = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111

Tabelul deplasari (nr. de shift-uri stanga per runda):
  1  1  2  2  2  2  2  2  1  2  2  2  2  2  2  1
```

```
C0 = 1111 0000 1100 1100 1010 1010 1111
D0 = 0101 0101 0110 0110 0111 1000 1111
C1  = 1110 0001 1001 1001 0101 0101 1111   (shift stanga cu 1)
D1  = 1010 1010 1100 1100 1111 0001 1110
C2  = 1100 0011 0011 0010 1010 1011 1111   (shift stanga cu 1)
D2  = 0101 0101 1001 1001 1110 0011 1101
C3  = 0000 1100 1100 1010 1010 1111 1111   (shift stanga cu 2)
D3  = 0101 0110 0110 0111 1000 1111 0101

Rezultat: C3 = 0000110011001010101011111111
          D3 = 0101011001100111100011110101
```

4.4 Task 2.3: round key K_i

For the same K^+ and $i = 1$:

```
K+ [56 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0xF0CCAAF556678F
i [1..16]: 1

C0 = 1111 0000 1100 1100 1010 1010 1111
D0 = 0101 0101 0110 0110 0111 1000 1111
C1  = 1110 0001 1001 1001 0101 0101 1111   (shift stanga cu 1)
D1  = 1010 1010 1100 1100 1111 0001 1110

Tabelul PC-2:
  14 17 11 24  1  5
   3 28 15  6 21 10
  23 19 12  4 26  8
  16  7 27 20 13  2
  41 52 31 37 47 55
  30 40 51 45 33 48
  44 49 39 56 34 53
  46 42 50 36 29 32

C1D1 (56 b): 1110000 1100110 0101010 1011111 1010101 0110011 0011110 0011110
K1 = PC-2(C1D1) (48 b): 000110 110000 001011 101111 111111 000111 000001 110010
K1 (hex): 1B02EFFC7072
```

4.5 Task 2.4: L_1 from an 8-character message

The message `Criptare` was entered:

```
Mesajul [8 caractere ASCII; 'r' = aleator]: Criptare

Mesaj in binar:
  'C' = 01000011
  'r' = 01110010
  'i' = 01101001
  'p' = 01110000
  't' = 01110100
  'a' = 01100001
  'r' = 01110010
  'e' = 01100101

Tabelul IP:
  58 50 42 34 26 18 10  2
  60 52 44 36 28 20 12  4
  62 54 46 38 30 22 14  6
  64 56 48 40 32 24 16  8
  57 49 41 33 25 17  9  1
  59 51 43 35 27 19 11  3
  61 53 45 37 29 21 13  5
  63 55 47 39 31 23 15  7

M        = 01000011 01110010 01101001 01110000 01110100 01100001 01110010 01100101
IP(M)    = 11111111 01011010 10010000 10100101 00000000 11111110 00000100 01000011
L0 = 11111111 01011010 10010000 10100101
R0 = 00000000 11111110 00000100 01000011

L1 = R0 = 00000000 11111110 00000100 01000011   (hex: 00FE0443)
```

4.6 Task 2.5: all 16 round keys

For the same K^+ (the table PC-2 and all C_i, D_i are printed first; only the final list is shown here):

```
Cheile de runda K1..K16:
K1  = 000110 110000 001011 101111 111111 000111 000001 110010   hex=1B02EFFC7072
K2  = 011110 011010 111011 011001 110110 111100 100111 100101   hex=79AED9DBC9E5
K3  = 010101 011111 110010 001010 010000 101100 111110 011001   hex=55FC8A42CF99
K4  = 011100 101010 110111 010110 110110 110011 010100 011101   hex=72ADD6DB351D
K5  = 011111 001110 110000 000111 111010 110101 001110 101000   hex=7CEC07EB53A8
K6  = 011000 111010 010100 111110 010100 000111 101100 101111   hex=63A53E507B2F
K7  = 111011 001000 010010 110111 111101 100001 100010 111100   hex=EC84B7F618BC
K8  = 111101 111000 101000 111010 110000 010011 101111 111011   hex=F78A3AC13BFB
K9  = 111000 001101 101111 101011 111011 011110 011110 000001   hex=E0DBEBEDE781
K10 = 101100 011111 001101 000111 101110 100100 011001 001111   hex=B1F347BA464F
K11 = 001000 010101 111111 010011 110111 101101 001110 000110   hex=215FD3DED386
K12 = 011101 010111 000111 110101 100101 000110 011111 101001   hex=7571F59467E9
K13 = 100101 111100 010111 010001 111110 101011 101001 000001   hex=97C5D1FABA41
K14 = 010111 110100 001110 110111 111100 101110 011100 111010   hex=5F43B7F2E73A
K15 = 101111 111001 000110 001101 001111 010011 111100 001010   hex=BF918D3D3F0A
K16 = 110010 110011 110110 001011 000011 100001 011111 110101   hex=CB3D8B0E17F5
```

4.7 Task 2.6: $B_1 \dots B_8$

For $i = 1$, $K_1 =$ `1B02EFFC7072` and $R_0 =$ `F0AAF0AA`:

```
Runda i [1..16]: 1
K1 [48 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x1B02EFFC7072
R0 [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0xF0AAF0AA

Tabelul E (expansiune 32 -> 48):
  32  1  2  3  4  5
   4  5  6  7  8  9
   8  9 10 11 12 13
  12 13 14 15 16 17
  16 17 18 19 20 21
  20 21 22 23 24 25
  24 25 26 27 28 29
  28 29 30 31 32  1

R0          = 11110000 10101010 11110000 10101010
E(R0)       = 011110 100001 010101 010101 011110 100001 010101 010101
K1            = 000110 110000 001011 101111 111111 000111 000001 110010
K1 + E(R0) = 011000 010001 011110 111010 100001 100110 010100 100111

B1B2...B8:
  B1 = 011000
  B2 = 010001
  B3 = 011110
  B4 = 111010
  B5 = 100001
  B6 = 100110
  B7 = 010100
  B8 = 100111
```

4.8 Task 2.7: $S_1(B_1) \dots S_8(B_8)$

For $B_1 \dots B_8 =$ `6117BA866527` (the eight S-box tables are printed first; only the lookups are shown):

```
Calcul S-box:
  B1=011000: rand=(00)=0, col=(1100)=12 -> S1= 5 = 0101
  B2=010001: rand=(01)=1, col=(1000)=8 -> S2=12 = 1100
  B3=011110: rand=(00)=0, col=(1111)=15 -> S3= 8 = 1000
  B4=111010: rand=(10)=2, col=(1101)=13 -> S4= 2 = 0010
  B5=100001: rand=(11)=3, col=(0000)=0 -> S5=11 = 1011
  B6=100110: rand=(10)=2, col=(0011)=3 -> S6= 5 = 0101
  B7=010100: rand=(00)=0, col=(1010)=10 -> S7= 9 = 1001
  B8=100111: rand=(11)=3, col=(0011)=3 -> S8= 7 = 0111

S1(B1)...S8(B8) = 0101 1100 1000 0010 1011 0101 1001 0111
                = 01011100100000101011010110010111   (hex: 5C82B597)
```

4.9 Task 2.8: R_k from L_{k-1} and the S-box output

For $k = 1$, $L_0 =$ `CC00CCFF` and S-box output `5C82B597`:

```
Runda k [1..16]: 1
L0 [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0xCC00CCFF
Rezultatul cutiilor S (32 biti) [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x5C82B597

Tabelul P:
  16  7 20 21 29 12 28 17
   1 15 23 26  5 18 31 10
   2  8 24 14 32 27  3  9
  19 13 30  6 22 11  4 25

S-out        = 01011100 10000010 10110101 10010111
P(S-out)     = 00100011 01001010 10101001 10111011
L0         = 11001100 00000000 11001100 11111111
R1 = L0 + P(S-out) = 11101111 01001010 01100101 01000100   (hex: EF4A6544)
```

4.10 Task 2.9: $S_j(B_j)$ for a given j

For $K_1 \oplus E(R_0) =$ `6117BA866527` and $j = 3$:

```
Ki + E(Ri-1) [48 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x6117BA866527
j (cutia S) [1..8]: 3

Ki + E(Ri-1) = 011000 010001 011110 111010 100001 100110 010100 100111
  B1 = 011000
  B2 = 010001
  B3 = 011110
  B4 = 111010
  B5 = 100001
  B6 = 100110
  B7 = 010100
  B8 = 100111

S3:
         0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  r=0   10  0  9 14  6  3 15  5  1 13 12  7 11  4  2  8
  r=1   13  7  0  9  3  4  6 10  2  8  5 14 12 11 15  1
  r=2   13  6  4  9  8 15  3  0 11  1  2 12  5 10 14  7
  r=3    1 10 13  0  6  9  8  7  4 15 14  3 11  5  2 12

B3 = 011110
rand   = primul+ultimul bit = 00 = 0
coloana= biti mijlocii      = 1111 = 15
S3(B3) = 8 = 1000
```

4.11 Task 2.10: R_i from the S-box output and L_{i-1}

```
Runda i [1..16]: 1
S1(B1)...S8(B8) (32 biti) [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x5C82B597
L0 [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0xCC00CCFF

Tabelul P:
  16  7 20 21 29 12 28 17
   1 15 23 26  5 18 31 10
   2  8 24 14 32 27  3  9
  19 13 30  6 22 11  4 25

S1(B1)...S8(B8) = 0101 1100 1000 0010 1011 0101 1001 0111
f = P(...)      = 00100011 01001010 10101001 10111011
L0            = 11001100 00000000 11001100 11111111
R1 = L0 + f = 11101111 01001010 01100101 01000100   (hex: EF4A6544)
```

4.12 Task 2.11: encrypted block from L_{16} and R_{16}

For $L_{16} =$ `43423234` and $R_{16} =$ `0A4CD995`:

```
L16 [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x43423234
R16 [32 biti, binar sau hex cu prefix 0x; 'r' = aleator]: 0x0A4CD995

Tabelul IP^-1 (permutarea finala):
  40  8 48 16 56 24 64 32
```

```
  39   7 47 15 55 23 63 31
  38   6 46 14 54 22 62 30
  37   5 45 13 53 21 61 29
  36   4 44 12 52 20 60 28
  35   3 43 11 51 19 59 27
  34   2 42 10 50 18 58 26
  33   1 41  9 49 17 57 25

L16 = 01000011 01000010 00110010 00110100
R16 = 00001010 01001100 11011001 10010101
R16L16 (se inverseaza!) = 00001010 01001100 11011001 10010101 01000011 01000010 00110010 00110100
C = IP^-1(R16L16)       = 10000101 11101000 00010011 01010100 00001111 00001010 10110100 00000101

Bloc criptat (hex): 85E813540F0AB405
```

The resulting block `85E813540F0AB405` is the known ciphertext of the test vector.

4.13 Complete encryption of a block

The block `01234567` was encrypted with the key `Cripto26`, with every round printed:

```
Blocul de mesaj [8 caractere ASCII; 'r' = aleator]: 01234567
Cheia [8 caractere ASCII; 'r' = aleator]: Cripto26
IP = 00000000 11111111 11110000 10101010 00000000 11111111 00000000 11001100
L0 = 00000000 11111111 11110000 10101010
R0 = 00000000 11111111 00000000 11001100
Runda  1: K=F0B62E4207A6 L=00FF00CC R=EFE33520
Runda  2: K=F0BEA614F1C2 L=EFE33520 R=4B1C8FAB
Runda  3: K=F0B67224A465 L=4B1C8FAB R=142FC04D
Runda  4: K=E4D676EAACC2 L=142FC04D R=184007CD
Runda  5: K=E6D3562CC71B L=184007CD R=E693B8C6
Runda  6: K=2ED3731F5442 L=E693B8C6 R=2DF4052F
Runda  7: K=AF517BCCC160 L=2DF4052F R=5976C84E
Runda  8: K=AF43D980EE4C L=5976C84E R=F09EA1BE
Runda  9: K=3F51DB411C59 L=F09EA1BE R=09E3EF67
Runda 10: K=3F49C98BB03C L=09E3EF67 R=F668E658
Runda 11: K=1B699D215FA4 L=F668E658 R=ADE16E85
Runda 12: K=1D2D9D1808B7 L=ADE16E85 R=31D40AAD
Runda 13: K=572CADC74895 L=31D40AAD R=0E3C687A
Runda 14: K=DBACA40323D9 L=0E3C687A R=4869D9A1
Runda 15: K=D8AEAEB39105 L=4869D9A1 R=03DC6CFF
Runda 16: K=E1AEA63F024E L=03DC6CFF R=9F342CE3

Bloc criptat: 11000011 11000011 01111110 01101110 01110010 00011111 00101011 01100011
Hex         : C3C37E6E721F2B63
```

4.14 Encryption and decryption of a text

The text `The Data Encryption Standard` (28 bytes, padded to 32) was encrypted with the key `Cripto26` and then decrypted:

```
Textul de criptat: The Data Encryption Standard
Cheia [8 caractere ASCII; 'r' = aleator]: Cripto26

Text criptat (binar, padding PKCS#7): 11000101 10101100 10101001 01010100 00110110 11010010 00000110
     01011010 01100011 10101001 01100001 00001001 01111000 10011100 10111111 01001010 00011001
    10000111 10101010 10010101 00100011 10100000 00011010 00101110 10111111 11010010 11000010
    11111001 01001000 01001100 11000001 11110101
Text criptat (hex):
    C5ACA95436D2065A63A96109789CBF4A1987AA9523A01A2EBFD2C2F9484CC1F5
```

```
Criptotextul [binar sau hex cu prefix 0x; 'r' = aleator]: 0
    xC5ACA95436D2065A63A96109789CBF4A1987AA9523A01A2EBFD2C2F9484CC1F5
Cheia [8 caractere ASCII; 'r' = aleator]: Cripto26

Text decriptat: The Data Encryption Standard
```

5 Conclusions

The program implements all the elements of DES required by the assignment and shows the tables and intermediate steps for each of them: the key schedule (K^+, C_i, D_i, K_i), the initial permutation, the expansion E, the S-boxes, the permutation P and the final permutation. The results match the classical worked example of DES at every step, and the full algorithm agrees with a reference library on 200 random tests.

Several properties of the algorithm became visible while working on the task. The eight parity bits of the key do not influence the result, because PC-1 discards them. The halves must be swapped before IP^{-1} (the `R16L16` line in task 2.11). Decryption reuses the encryption network with the round keys in reverse order. Since the text mode encrypts blocks independently (ECB), equal plaintext blocks give equal ciphertext blocks, and a 56-bit key is too short for real use; therefore the program should be used only for learning.

A limitation of the implementation is that the alternative one-round variant from the note of the assignment is not a separate menu option; it can be obtained by calling `des_encrypt_block` with `rounds=1`.

6 Source Code

GitHub repository: `https://github.com/RodionBm/Cryptography`$_{l}$*abs*/*tree*/*main*/*Cryptolab*4